LMS - Learning Management System

Group members: S. Gaponenko, A. Abdrakhmanova
GitHub: https://github.com/GestasContrad/SDF_Final

Topic: Learning Management System. The system is designed to automate interactions between teachers and students: creating courses, assigning assignments, and checking solutions.

I. Functional Requirements Mapping:

- **FR-1:** Role-Based Access Control: Supports user registration and authorization with division into roles (Student, Teacher, Admin). Implemented via the User entity and Spring Security.
- **FR-2:** Course & Lesson Management: Teachers have the right to create courses and attach lessons to them. Supported via controllers with TEACHER role checks.
- **FR-3:** Assignment System: Teachers can create and attach assignments to specific lessons.
- **FR-4:** Submissions Handling: Students can view available assignments and upload their submissions.
- **FR-5:** Grading & Mock Notifications Module: The instructor can assign a grade (from 0 to 100) for submitted assignments, after which the system simulates sending a notification via the Mock service.

II. Non-Functional requirements Mapping:

- **NFR-1:** Performance: API response time for basic CRUD operations (reading courses, viewing lessons) should not exceed 500 ms for the 95th percentile (p95) under normal load.
- **NFR-2:** Scalability: The system should reliably handle up to 1,000 concurrent connections. The base load is 5-10 RPS (requests per second), and during peak periods (deadlines), it should withstand surges of up to 50 RPS without service degradation.

- **NFR-3:** Security: Use stateless JWT tokens for authentication. All passwords in the database should be hashed using the BCrypt algorithm, and access to endpoints should be protected at the role level.

- **NFR-4:** Reliability and Integrity: Strict database schema versioning via Liquibase to prevent data loss. ACID transactions for grading and submissions.

- **NFR-5:** Portability: Full stack containerization (Spring Boot + PostgreSQL) via Docker and docker-compose for single-command deployment with environment variable forwarding (.env).

III. Resource Estimation & Scope Boundaries:

User load calculation:

- **Audience:** Approximately 5,000 registered students.

- **Daily activity (DAU):** ~500 unique users per day

- **Traffic:** Average 5-10 requests per second, up to 50 RPS at peak.

Data Capacity Planning:

- **Relational Data (DB):** Users, course associations, and assessments will generate approximately 1-2 GB of text data in the first year of operation.

- **File Storage (Optional):** If 500 active students submit 10 assignments up to 5 MB each, an additional ~25 GB of space will be required per year (500 x 10 x 5 = 25,000 MB).

- **Final Storage Requirements:** A 50 GB SSD will be more than sufficient to cover the needs of the DB and operating system, considering overhead.

Hardware Requirements (Server Specs) and Architectural Boundaries:

Computing Power: For stable operation of the JVM (Spring Boot) and PostgreSQL in a Docker environment, a server with 2 vCPUs and 4-8 GB of RAM is required.

Engineering Rationale: The Spring Boot container is allocated 1-2 GB of heap memory. The remaining RAM will be used by PostgreSQL for index caching, allowing it to efficiently handle peak loads of 50 RPS without the need to immediately implement external caching systems like Redis or load balancers. External calls are isolated by integration with httpbin / MockServer, preventing threads from blocking while waiting for a response from the mail gateway.

IV. Testing Strategy & Quality Assurance:

Unit Testing (Mockito & JUnit 5)

- **Goal:** Isolated testing of the service layer (Business Logic) without raising the Spring context or accessing the real database.

- **Implementation:** Using the Mockito library, mocks are created for the Repository layers. This allows for edge case testing. For example, when testing the evaluation method (gradeSubmission), we artificially simulate returning data from the database and check whether the system throws an exception when attempting to assign a grade greater than 100 or less than 0. This ensures consistent compliance with business rules.

Data JPA Slice Testing (@DataJpaTest)

- **Goal:** Isolated testing of the data access layer (Repository Layer) and verification of the correctness of the object-relational mapping (ORM/JPA).

- **Implementation:** Using the @DataJpaTest annotation, Spring Boot bootstraps only JPA-related components (entities, repositories, EntityManager). Tests are run on a real PostgreSQL database deployed in a Docker container via Testcontainers. These tests verify:

The correctness of Spring Data JPA custom queries (e.g., searching for lessons by courseId).

The entity lifecycle and cascade operations (e.g., verifying that deleting a course from the database automatically deletes all associated lessons and assignments).

Enforcement of database constraints (saving a duplicate email address should throw a DataIntegrityViolationException).

Full Integration Testing (@SpringBootTest)

- **Goal:** End-to-end testing of interactions across all application layers (Controller -> Security -> Service -> Repository -> Database) under production-like conditions.

- **Implementation:** @SpringBootTest abstraction bootstraps the full application context. The PostgreSQL database is isolated using Testcontainers and automatically updated using Liquibase migration scripts before starting tests. Integration tests verify:

The operation of SpringSecurity security filters (verifying that the assessment endpoint actually returns a 403 Forbidden status for the student token).

The correct serialization/deserialization of JSON packets and the operation of MapStruct mappers.

Successful execution of distributed ACID transactions in end-to-end scenarios (e.g., user registration -> login -> course creation).

Code Coverage Measurement (Jacoco)

- **Goal:** Analyze the completeness of source code coverage by automated tests.

- **Implementation:** The Jacoco plugin is integrated into build.gradle. Running the ./gradlew jacocoTestReport command generates a detailed HTML report showing the code coverage percentage by classes, methods, and lines. The primary focus of coverage is on the Service and Security layers, as they contain the most critical system algorithms.

API Verification & Manual Execution (Postman)

- **Goal:** End-to-end testing of client scripts and verification of API contracts.

- **Implementation:** A collection of requests (Postman Collection) is stored in the repository (in the /postman folder). It includes automatic scripts (Pre-request Scripts and Tests) for saving the JWT token after login to Postman environment variables, allowing instructors and assessors to instantly test secure endpoints without manually copying tokens.

V. Externalized Configuration & Secrets Management (.env):

In accordance with the Twelve-Factor App methodology for developing modern cloud applications, all system configurations containing sensitive data or varying across runtime environments are strictly isolated from the source code. Sensitive values are no longer hardcoded within the project, which significantly improves security by eliminating the risk of leaking real passwords into public GitHub repositories.

Rationale & Implementation:

To manage secrets securely, the project utilizes an .env (Environment Variables) file located in the root directory and explicitly excluded from version control via .gitignore. This file stores critical parameters, including:

- Database credentials (DB_USER, DB_PASSWORD).
- The cryptographic key for token signing and validation (JWT_SECRET).
- Network ports for Docker containers (SERVER_PORT, DB_PORT).
- Integration Mechanism (Engineering Value):

Docker Compose Integration: The docker-compose utility automatically reads variables from the .env file in the project root and injects them into the PostgreSQL and Spring Boot containers during the docker-compose up execution.

Spring Boot Externalization: The application.properties file dynamically resolves these values at runtime rather than storing raw text. For example:

- spring.datasource.password=${DB_PASSWORD}
- application.security.jwt.secret-key=${JWT_SECRET}

Trade-off & Team Enablement:

While externalizing configuration requires manually creating the .env file when deploying to a new machine, the security benefits far outweigh this trade-off. To ensure seamless onboarding for reviewers and new developers, a safe template named .env.example is provided in the repository. It outlines the required variable structure with secure default values for local development, allowing a quick setup without compromising production secrets.

VI. High-Level Design (HLD) & System Architecture:

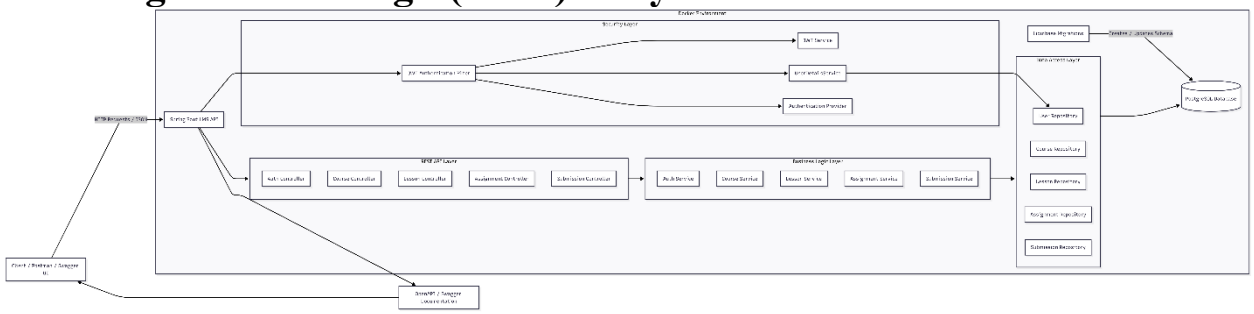

Figure 1: High-Level System Architecture and Container Boundaries

System Architecture Overview

Figure 1 illustrates the High-Level Design (HLD) of the Learning Management System, demonstrating a clear separation of concerns across multiple network layers. The architecture follows a classic multi-tier backend model, completely isolated within Docker containers.

Request Routing Flow:

Client Layer: Clients (such as Postman or a web browser) send HTTP REST requests (e.g., POST /api/auth/login or GET /api/courses).

API Controller Layer: The Spring Boot API Gateway intercepts these requests. It handles input validation, sanitization, and maps JSON payloads to internal Data Transfer Objects (DTOs) using MapStruct.

Service Layer (Business Logic): This tier contains the core domain logic (e.g., grading boundaries, role verification). It ensures that database interactions are strictly separated from HTTP protocols.

Data Access & Storage: The Repository layer leverages Spring Data JPA to execute queries against the PostgreSQL database. PostgreSQL operates in an isolated Docker container on port 5432.

External Integrations: The Service layer communicates with external mock services (like MockServer/httpbin) to simulate email notifications without blocking main processing threads.

This multi-container orchestration ensures scalability (NFR-2) and portability (NFR-5), allowing independent scaling of the JVM process and the database engine.

VII. Database Modeling & ERD:

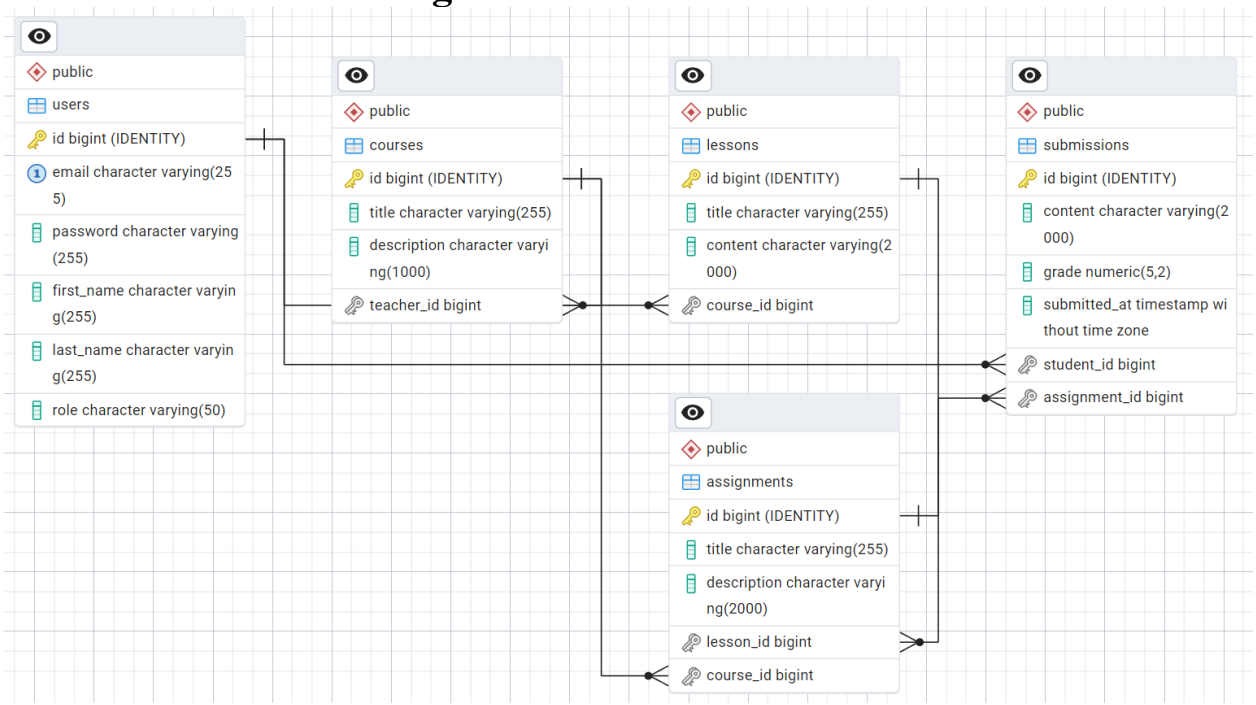

Figure 2: Entity-Relationship Diagram (ERD) mapping core LMS tables (Normalized to 3NF)

Database Schema & Relational Modeling

Figure 2 presents the logical schema of the underlying PostgreSQL database, utilizing standard Crow's Foot notation to define exact multiplicities and foreign key relationships. The schema is normalized to the Third Normal Form (3NF) to eliminate data redundancy and ensure relational integrity.

Key Entities & Associations:

users Table: Stores authentication credentials (BCrypt hashed passwords) and RBAC roles (STUDENT, TEACHER, ADMIN).

courses & lessons Tables: Exhibit a One-to-Many relationship (1:N). A course acts as an aggregate root containing multiple lessons.

assignments & submissions Tables: Map the core evaluation flow. An assignment belongs to a specific lesson. Students create submissions linked via foreign keys to both the assignments table and the users table (as the author).

Data Integrity: All foreign keys (FK) strictly enforce cascading rules where applicable (e.g., deleting a course deletes its lessons). Indexing is applied to frequently queried foreign keys (like course_id inside lessons) to meet the performance metric (NFR-1) of resolving queries within 500 ms under load. Chema evolution is safely managed via Liquibase.

VIII. LLD / Sequence Diagram: Login and JWT Generation:

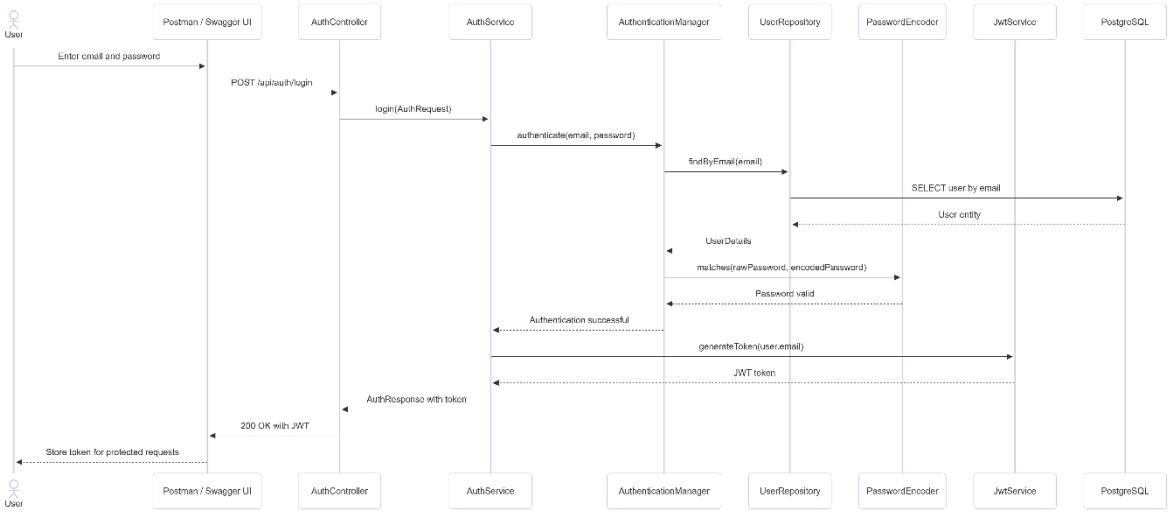

Figure 4: UML Sequence Diagram — Authentication and Stateless JWT Generation

Component Interaction: Security & Authentication

Figure 4 breaks down the object-oriented logic for user authentication, fulfilling strict security constraints (NFR-3).

Execution Loop:

1. The Client submits a POST /api/auth/login request with an email and plain-text password.

2. The AuthController delegates the credentials to the AuthenticationManager of Spring Security.

3. The CustomUserDetailsService queries the UserRepository to find the user record by email.

4. The BCryptPasswordEncoder intercepts the retrieved hash and mathematically verifies it against the provided plain-text password.

5. If successful, the JwtTokenProvider mathematically signs a new stateless JWT token (encoding the user's ID, Role, and expiration date) using the secure server secret.

6. The generated token is returned to the client inside an AuthResponseDTO.

This stateless pattern eliminates the need for server-side session memory, allowing the Spring Boot container to scale horizontally without session synchronization bottlenecks.

IX. LLD / Sequence Diagram: Grade Submission:

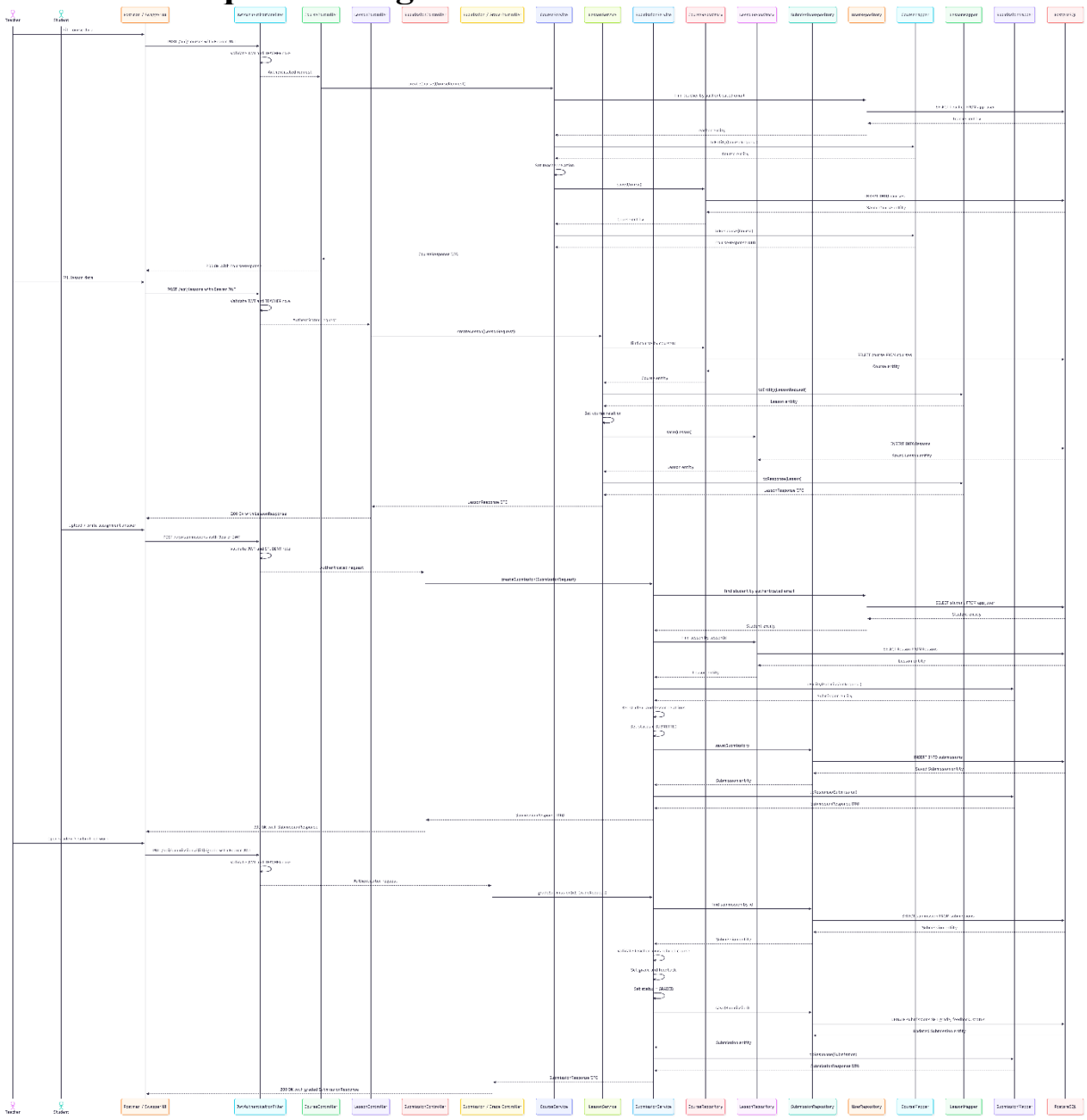

Figure 3: UML Sequence Diagram — Student Grade Submission and Evaluation Lifecycle

Component Interaction: Grade Submission Flow

Figure 3 details the low-level chronological method invocation lifecycle for one of the system's most critical operations (FR-4 and FR-5): submitting and grading an assignment.

Execution Loop:

1. The Teacher client sends a PUT /api/submissions/{id}/grade request containing the grade payload (0-100).

2. The SubmissionController intercepts the request and verifies the JWT token for the TEACHER role authority.

3. The controller delegates the payload to the SubmissionService.gradeSubmission() method.

4. The service fetches the entity from SubmissionRepository, applies grading boundary validations, updates the entity state, and triggers an ACID transaction commit to PostgreSQL.

5. Upon successful persistence, the service asynchronously dispatches an HTTP request to an external Mock Notification Service to simulate emailing the student.

6. The updated entity is mapped back to a SubmissionResponseDTO and returned to the client with a 200 OK HTTP status code.

X. Engineering Justification & Trade-off Analysis:

Backend Framework: Java 21 & Spring Boot 3.4

Rationale: Spring Boot was chosen due to its robust ecosystem, built-in support for Inversion of Control (IoC), and integration with Spring Security. Switching to Java 21 enables the use of virtual threads and pattern matching, which optimizes API performance under high loads (NFR-1, NFR-2).

Tradeoff: JVM applications consume more RAM than Go or Node.js solutions. However, this drawback is offset by high development speed, strong typing, and enterprise-grade reliability, which are critical for an educational platform.

Relational Database: PostgreSQL

Rationale: LMS data (Courses, Lessons, Assignments, Users) has a strict hierarchical and relational structure. PostgreSQL provides full support for ACID transactions. This ensures that the grading and submission processes (Submissions) do not result in data loss or race conditions during concurrent requests.

Trade-off: Relational databases are more difficult to scale horizontally compared to NoSQL databases (e.g., MongoDB). However, NoSQL was rejected because the lack of strong foreign keys and join operations would complicate maintaining data integrity when aggregating student grades.

Security & Authentication: Stateless JWT (JSON Web Tokens)

Rationale: Using JWT satisfies the security requirement (NFR-3). The token-based authentication architecture is stateless. The server does not need to store user sessions in RAM or a database.

Trade-off: A disadvantage of JWT is the difficulty of forcibly revocation of a token before its expiration date. To minimize this risk, we do not store sensitive data within the token payload and set a reasonable expiration time for the key. This approach allows for easy horizontal scaling of the API in the future without the need to implement Redis for session storage.

Infrastructure & Data Integrity: Docker & Liquibase

Rationale (Docker): Full containerization (NFR-5) ensures that the application will run identically on developer machines, in CI/CD pipelines, and on the production server.

Rationale (Liquibase): Using Liquibase satisfies NFR-4 (Data Reliability). Instead of manually executing SQL scripts, the database schema is versioned along with the code (in changelog format). This prevents database degradation when deploying new features and allows for easy rollback of changes in the event of a failure.

XI. API Specifications & Contract Definitions:

This section contains specifications for the key REST API contracts. A complete interactive specification of all existing system endpoints is automatically generated and available in the Swagger UI.

Interactive API Documentation (OpenAPI / Swagger UI)

The springdoc-openapi-ui library (OpenAPI 3.0 standard) is integrated into the project for automatic documentation generation.

Local URL for access: http://localhost:8080/swagger-ui/index.html

Engineering Value: The specification is updated automatically when the Java code changes, eliminating desynchronization. The interface allows front-end developers and testers to test endpoints in real time. Protected endpoints require one-time authorization via the "Authorize" button (in the Bearer <token> format).

Core Endpoints

User authentication and token issuance

URL: /api/auth/login

HTTP method: POST

Access level: Public (Available to everyone)

Description: Verifies user credentials and generates a cryptographically signed stateless JWT token for subsequent secure requests.

Request Body (JSON):

JSON

```
{
"email": "teacher@lms.edu",
"password": "SecurePassword123"
}
```

Field validation rules: email must match a valid email format (@Email), both fields must not be empty (@NotBlank).

Successful response (Response 200 OK):

JSON

```
{
"email": "teacher@lms.edu",
"role": "TEACHER",
"token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ0ZWFjaGVyQGxtcy5lZHUiLCJyb2xlIjoiVEVBQ0hFUiIsImlhdCI6MTcxNjUwMDAwMCwiZXhwIjoxNzE2NTg2NDAwfQ..."
}
```

Error codes: 401 Unauthorized (invalid credentials), 400 Bad Request (invalid JSON).

Grading a Student's Submission (Main Business Logic)

URL: /api/submissions/{submissionId}/grade

HTTP Method: PUT

Access Level: Protected (Requires the Authorization: Bearer <token> header and the TEACHER role)

Description: Assigns a grade to a student's solution, verifies business boundaries, and initiates an asynchronous email notification simulation.

Path Parameters: submissionId (Long) — the unique ID of the work being graded.

Request Body (JSON):

JSON

```
{
"grade": 95,
"feedback": "Excellent test coverage and proper database migration schema using Liquibase."
}
```

Field validation rules: grade — a required integer between 0 and 100, strictly inclusive (annotations @Min(0) and @Max(100)). feedback — a string of up to 500 characters.

Successful response (Response 200 OK):

JSON

```
{
"submissionId": 102,
"studentId": 1,
"assignmentId": 34,
"grade": 95,
"feedback": "Excellent test coverage and proper database migration schema using Liquibase.",
"gradedAt": "2026-05-24T14:00:00Z",
"status": "GRADED",
"notificationSent": true
}
```

*Error codes: * 400 Bad Request — grade outside the range 0–100.*

401 Unauthorized — token not transferred or expired.

403 Forbidden — The request was sent by a student or instructor who is not the course author.

404 Not Found — A job with this ID was not found in the database.